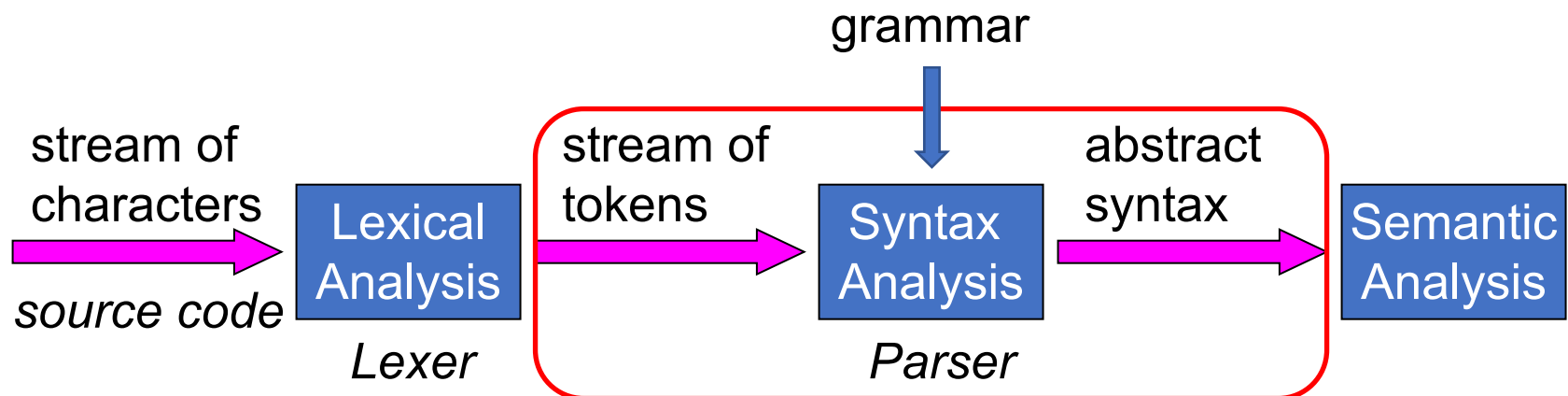


Parsing

Chapter 3

What is syntax analysis?

- **Syn-tax**: the way in which words are put together to form phrases, clauses, or sentences.
- **Syntax analysis**: parsing the phrase structure of the program



The front-end of a compiler.

- The parser is (manually/automatically) constructed based on the grammar.

Why do we need syntax analysis?

- Check the syntax of the input source code:

```
1 int foo(int x){  
2   int y;  
3   if ((x > 0)  
4     y = 1;  
5   else  
6     y = 0  
7   return y;  
8 }
```

foo.c:4:9: **error:** expected ')'

y = 1;
^

foo.c:3:8: **note:** to match this '('

if ((x > 0)
^

foo.c:6:14: **error:** expected ';' after expression

y = 0
^

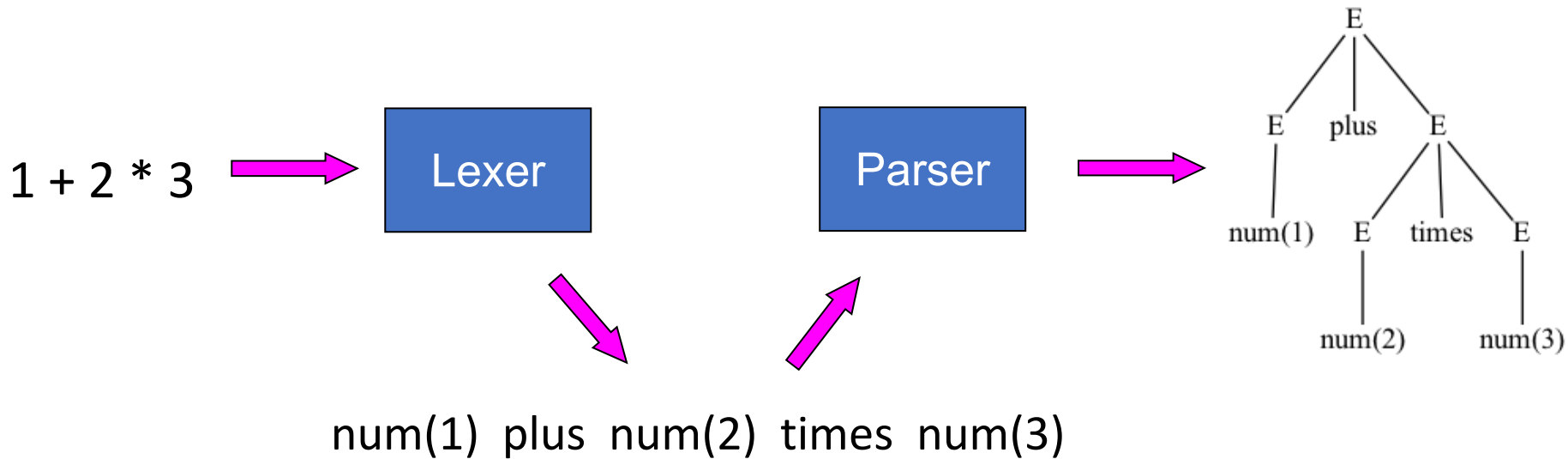
;

2 errors generated._

- No lexical errors, but multiple syntax errors.
- Why not check syntax errors during semantic analysis?
 - Assume no syntax errors during semantic analysis

Why do we need syntax analysis?

- Construct a parse tree and ease the following phases:



- The expression can be correctly evaluated based on the parse tree.

Outline: How to build a parser?

- Specifying the syntax of a programming language with **Context-Free Grammars (CFG)**
- Build the parser based on the CFG:
 - **Top-Down Parsing**
 - Predictive Parsing
 - **Bottom-Up Parsing**
- More about parsing:
 - **Automatic parser generation**
 - **Error Recovery**

3.1 Context-Free Grammar

Outline: How to build a parser?

- **Specifying the syntax of a programming language with Context-Free Grammars (CFG)**
- Build the parser based on the CFG:
 - Top-Down Parsing
 - Recursive Descent Parsing
 - Predictive Parsing (LL(k) Parsing)
 - Bottom-Up Parsing
- More about parsing:
 - Automatic parser generation
 - Error Recovery

Context-Free Grammars

- Not all strings of tokens are programs.
- A parser must distinguish between valid and invalid strings of tokens.
- We need:
 - Formally describe valid strings of tokens
 - A method for distinguishing valid from invalid strings of tokens
- A **language**: a set of **strings**, each string is a finite sequence of **symbols** taken from a finite **alphabet**
- For parsing: **symbols** are **lexical tokens**

Context-Free Grammars

- Regular languages
 - The weakest formal languages widely used
- Consider the following language:

$$\{(^i)^i \mid i \geq 0\}$$



Recursive Structure

$$\{\varepsilon, (), (()), ((())), \dots\}$$

- This language is not regular $\rightarrow$ cannot be recognized by a regular expression
- Recursive structure is very common in lots of programming languages
 - $((1+2) + 3)$

**We need more powerful formal languages
to specify programming languages.**

Context-Free Grammars

- Context-free grammars (CFG) are a natural notion for recursive structure
 - A Context-Free Grammar (CFG) consists of:
 - A set of terminals T : symbols from the alphabet
 - A set of non-terminals N
 - A start symbol S , $S \in N$
 - A set of productions (rules)
 - $X \rightarrow Y_1 Y_2 \dots Y_k$ replace X with $Y_1 Y_2 \dots Y_k$
- $X \in N \quad Y_i \in N \cup T \cup \{\varepsilon\}$

$\{(i)^i \mid i \geq 0\} \quad \rightarrow \quad \begin{array}{l} S \rightarrow (S) \\ S \rightarrow \varepsilon \end{array}$

$T = \{ (,) \}$

$N = \{ S \}$

S

2 productions

Context-Free Grammars

- Derive strings based on a CFG:
 - Begin with a string with only the start symbol S
 - Replace any non-terminal X in the string by the right-hand side of some production $X \rightarrow Y_1 \dots Y_k$
 - Repeat (2) until there are only terminals in the string

$S \rightarrow (S)$	$\longrightarrow$	$S \rightarrow (S)$
$S \rightarrow \epsilon$		$(S) \rightarrow ((S))$
		$((S)) \rightarrow (())$

- More formally, at each step:

Current state:

$X_1 \dots X_i X X_{i+1} \dots X_N$

Next state:

$X_1 \dots X_i Y_1 \dots Y_k X_{i+1} \dots X_N$

Some production:

$X \rightarrow Y_1 \dots Y_k$

Context-Free Grammars

- Let G be a context-free grammar with start symbol S . Then the language $L(G)$ of G is:

$$L(G) = \{a_1 \dots a_n \mid \forall_i a_i \in T \wedge S \xrightarrow{*} a_1 \dots a_n\}$$

- $*$ means zero or more steps

$$\begin{array}{c} S \rightarrow \dots \rightarrow \dots \rightarrow \alpha_0 \rightarrow \alpha_1 \rightarrow \alpha_2 \rightarrow \dots \rightarrow \alpha_n \\ \downarrow \\ \alpha_0 \xrightarrow{*} \alpha_n \end{array}$$

- Once generated, terminals are permanent
- For parsing, terminals ought to be **lexical tokens** of the language

Context-Free Grammars

- | | | |
|---------------------------------------|-------------------------------|-------------------------|
| 1. $S \rightarrow S; S$ | 4. $E \rightarrow \text{id}$ | 8. $L \rightarrow E$ |
| 2. $S \rightarrow \text{id} := E$ | 5. $E \rightarrow \text{num}$ | 9. $L \rightarrow L, E$ |
| 3. $S \rightarrow \text{print} (L)$ | 6. $E \rightarrow E + E$ | |
| | 7. $E \rightarrow (S, E)$ | |

Grammar 3.1. A syntax for straight-line programs.

- $T = \{;, \text{id}, :=, \text{print}, \text{num}, +, (,), ", "\}$
- $N = \{S, E, L\}$
- Start symbol: S
- 9 productions

Strings in the language of this CFG:

$\text{id} := \text{num}$

$\text{id} := \text{num} + \text{num}$

$\text{print} (\text{num})$

$\text{id} := \text{id} + (\text{id} := \text{num} + \text{num}, \text{id})$

.....

source code can be:

$a := 7$

$b := c + (d := 5 + 6, d)$

.....

Context-Free Grammars - Derivations

- We need
 - ~~Formally describe valid strings of tokens~~
 - A method for distinguishing valid from invalid strings of tokens -> decide whether a string is in the language of a CFG
- A **derivation**: start with the start symbol S , then repeatedly replace any non-terminal by one of its right-hand sides
$$S \rightarrow \dots \rightarrow \dots \rightarrow \dots \rightarrow \dots$$
a sequence of productions
- A string is in $L(G)$ iff it is **possible** to **derive** that string from the start symbol S .

Context-Free Grammars - Derivations

- Example:

1. $E \rightarrow E * E$

2. $E \rightarrow E / E$

3. $E \rightarrow E + E$

4. $E \rightarrow E - E$

5. $E \rightarrow \text{id}$

6. $E \rightarrow \text{num}$

7. $E \rightarrow (E)$

Grammar 3.5.

id * id + id

E

E + E

E * E + E

id * E + E

id * id + E

id * id + id

Context-Free Grammars - Derivations

- A derivation can be drawn as a tree
 - Start symbol is the tree's root
 - For a production $X \rightarrow Y_1 \dots Y_k$, add children $Y_1 \dots Y_k$ to node X

id * id + id

E

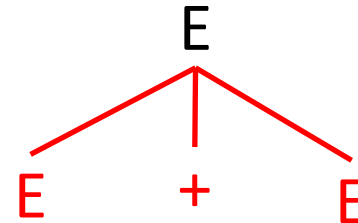
E

Context-Free Grammars - Derivations

- A derivation can be drawn as a tree
 - Start symbol is the tree's root
 - For a production $X \rightarrow Y_1 \dots Y_k$, add children $Y_1 \dots Y_k$ to node X

id * id + id

E
E + E

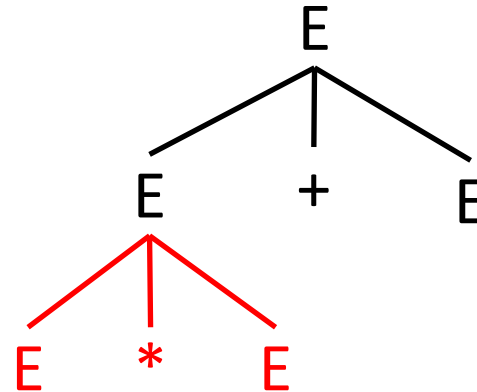


Context-Free Grammars - Derivations

- A derivation can be drawn as a tree
 - Start symbol is the tree's root
 - For a production $X \rightarrow Y_1 \dots Y_k$, add children $Y_1 \dots Y_k$ to node X

id * id + id

E
E + E
E * E + E

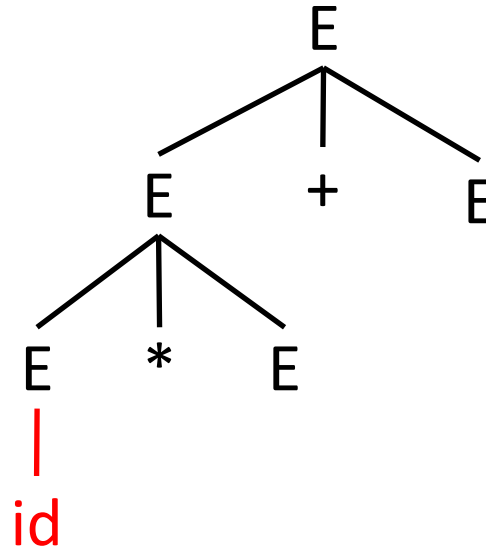


Context-Free Grammars - Derivations

- A derivation can be drawn as a tree
 - Start symbol is the tree's root
 - For a production $X \rightarrow Y_1 \dots Y_k$, add children $Y_1 \dots Y_k$ to node X

id * id + id

E
E + E
E * E + E
id * E + E



Context-Free Grammars - Derivations

- A derivation can be drawn as a tree
 - Start symbol is the tree's root
 - For a production $X \rightarrow Y_1 \dots Y_k$, add children $Y_1 \dots Y_k$ to node X

id * id + id

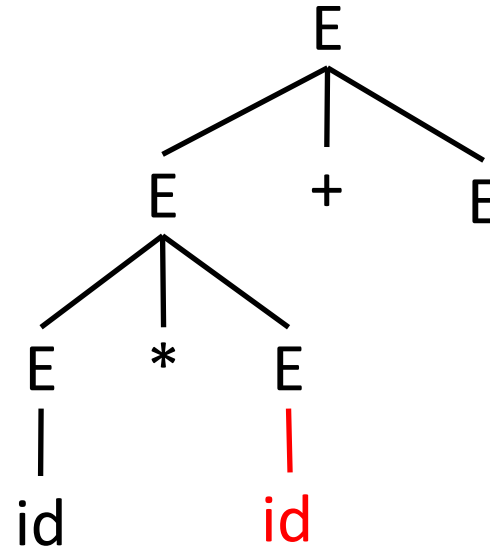
E

E + E

E * E + E

id * E + E

id * id + E



Context-Free Grammars - Derivations

- A derivation can be drawn as a tree
 - Start symbol is the tree's root
 - For a production $X \rightarrow Y_1 \dots Y_k$, add children $Y_1 \dots Y_k$ to node X

id * id + id

E

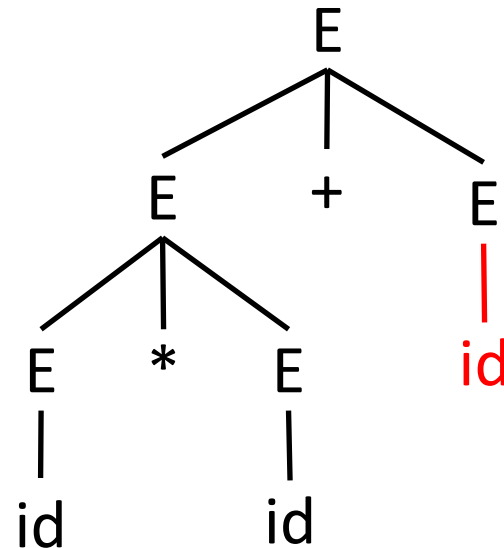
E + E

E * E + E

id * E + E

id * id + E

id * id + id

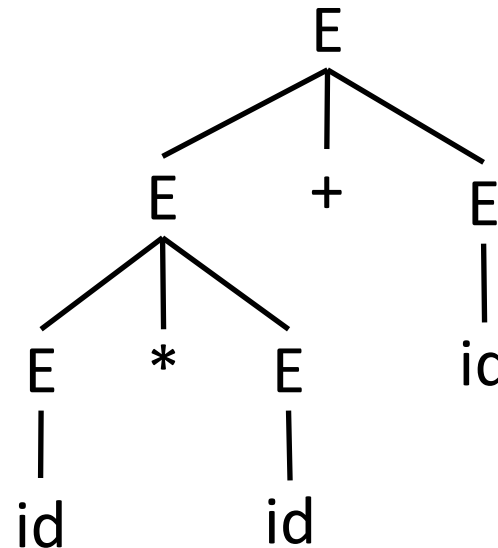


Context-Free Grammars - Derivations

- A derivation can be drawn as a tree
 - Start symbol is the tree's root
 - For a production $X \rightarrow Y_1 \dots Y_k$, add children $Y_1 \dots Y_k$ to node X

id * id + id

E
E + E
E * E + E
id * E + E
id * id + E
id * id + id



Parse Tree

Context-Free Grammars - Derivations

E

E + E

E * E + E

id * E + E

id * id + E

id * id + id

- The example is a **left-most derivation**
 - at each step, replace the left-most non-terminal
- There is an equivalent notion of a **right-most derivation**

Context-Free Grammars - Derivations

- A derivation can be drawn as a tree
 - Start symbol is the tree's root
 - For a production $X \rightarrow Y_1 \dots Y_k$, add children $Y_1 \dots Y_k$ to node X

id * id + id

E

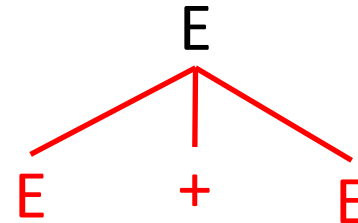
E

Context-Free Grammars - Derivations

- A derivation can be drawn as a tree
 - Start symbol is the tree's root
 - For a production $X \rightarrow Y_1 \dots Y_k$, add children $Y_1 \dots Y_k$ to node X

id * id + id

E
E + E

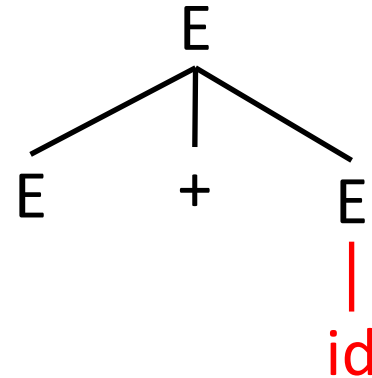


Context-Free Grammars - Derivations

- A derivation can be drawn as a tree
 - Start symbol is the tree's root
 - For a production $X \rightarrow Y_1 \dots Y_k$, add children $Y_1 \dots Y_k$ to node X

id * id + id

E
E + E
E + id

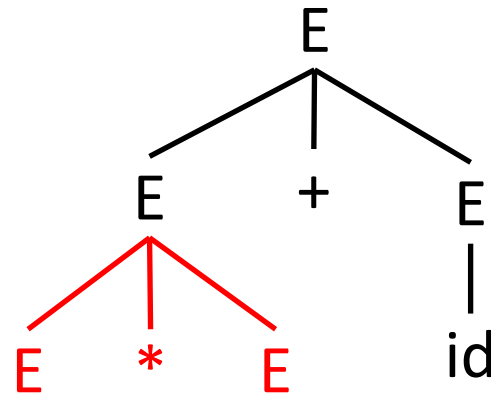


Context-Free Grammars - Derivations

- A derivation can be drawn as a tree
 - Start symbol is the tree's root
 - For a production $X \rightarrow Y_1 \dots Y_k$, add children $Y_1 \dots Y_k$ to node X

id * id + id

E
E + E
E + id
E * E + id



Context-Free Grammars - Derivations

- A derivation can be drawn as a tree
 - Start symbol is the tree's root
 - For a production $X \rightarrow Y_1 \dots Y_k$, add children $Y_1 \dots Y_k$ to node X

id * id + id

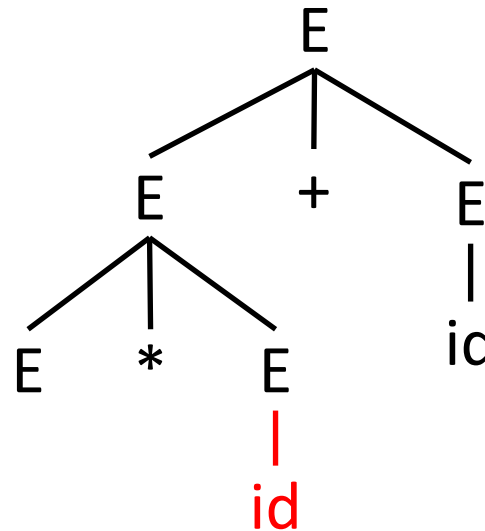
E

E + E

E + id

E * E + id

E * id + id



Context-Free Grammars - Derivations

- A derivation can be drawn as a tree
 - Start symbol is the tree's root
 - For a production $X \rightarrow Y_1 \dots Y_k$, add children $Y_1 \dots Y_k$ to node X

id * id + id

E

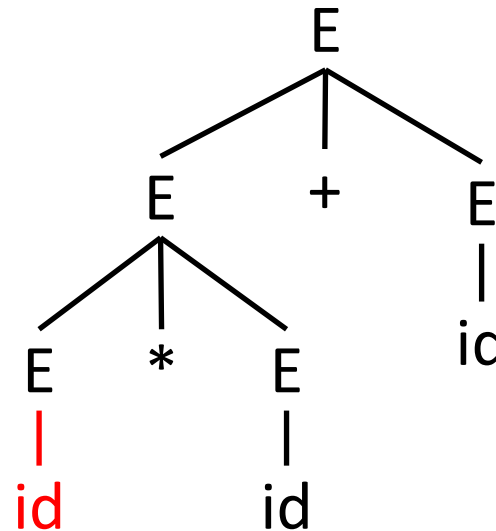
E + E

E + id

E * E + id

E * id + id

id * id + id



Context-Free Grammars - Derivations

- A derivation can be drawn as a tree
 - Start symbol is the tree's root
 - For a production $X \rightarrow Y_1 \dots Y_k$, add children $Y_1 \dots Y_k$ to node X

id * id + id

E

E + E

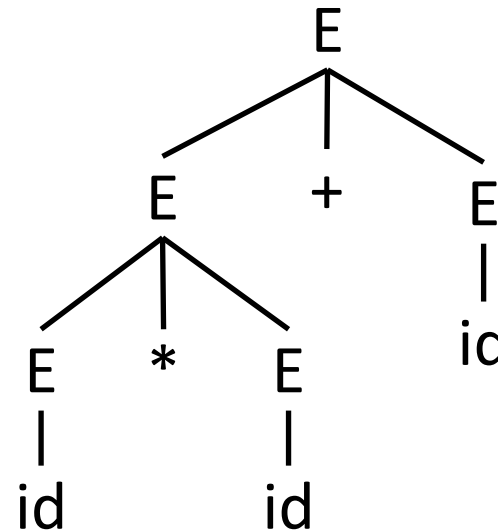
E + id

E * E + id

E * id + id

id * id + id

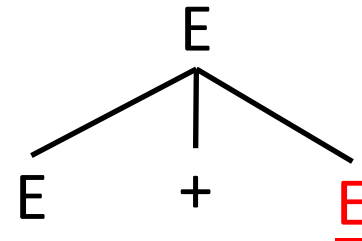
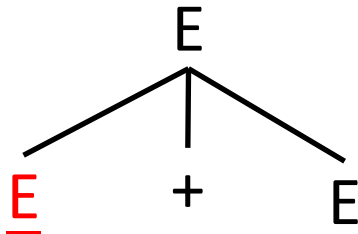
Right-Most Derivation



Parse Tree

Context-Free Grammars - Derivations

- The left-most and the right-most derivations have the same parse tree.
 - This is not accidental.



- There can be some derivations that are neither left-most nor right-most

$\underline{E}$

$\underline{E} + E$

$E * \underline{E} + E$

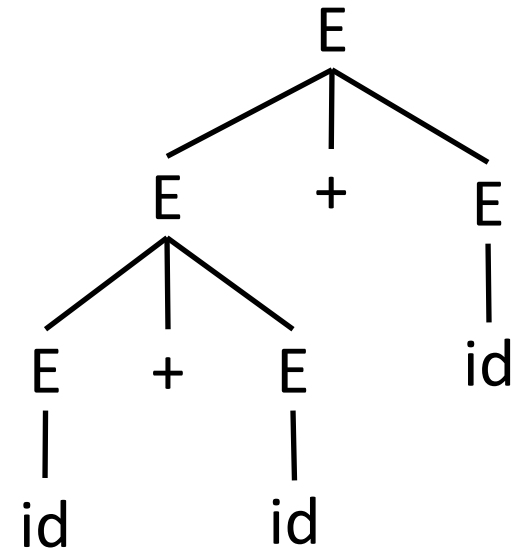
$\underline{E} * id + E$

$id * id + \underline{E}$

$id * id + id$

Context-Free Grammars - Parse Tree

- A parse tree has
 - Terminals at the leaves
 - Non-terminals at the interior nodes
- An in-order traversal of the leaves is the original input
- The parse tree shows the association of operations, the input string does not.



Context-Free Grammars - Parse Tree

- We are not just interested in whether $s \in L(G)$
 - We need a parse tree for s
 - We can know the phrase structure, the association of operations of operations
- A derivation defines a parse tree
 - But one parse tree may have many derivations
- Left-most and right-most derivations are important in parser implementation
 - For an unambiguous CFG, the leftmost and rightmost derivations are unique.

Context-Free Grammars – Ambiguous Grammars

- Grammar:

1. $E \rightarrow E * E$

2. $E \rightarrow E / E$

3. $E \rightarrow E + E$

4. $E \rightarrow E - E$

5. $E \rightarrow \text{id}$

6. $E \rightarrow \text{num}$

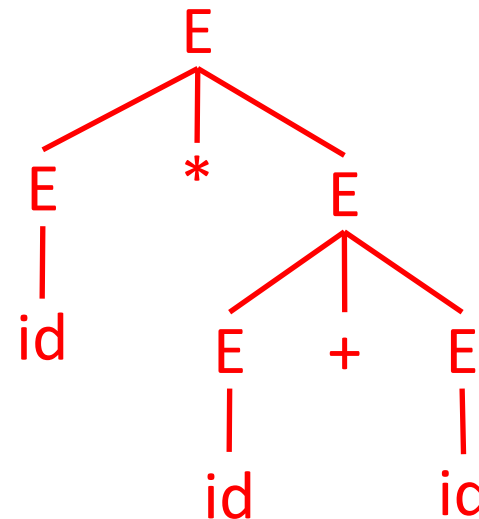
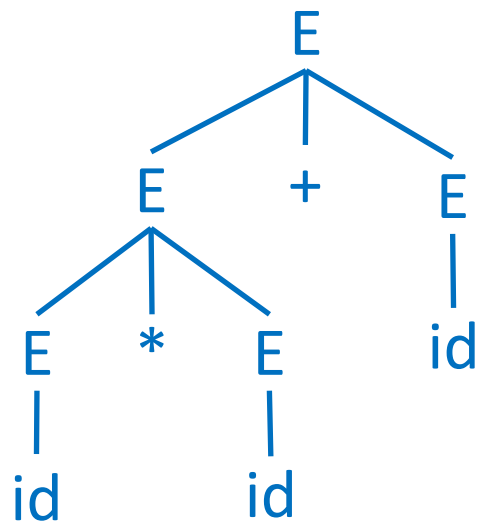
7. $E \rightarrow (E)$

Grammar 3.5.

- String:

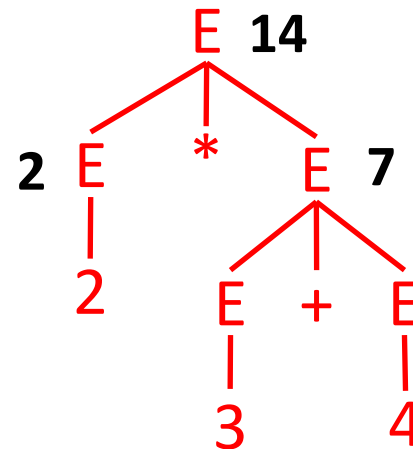
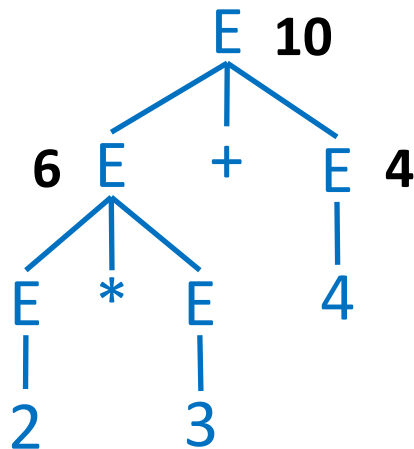
id * id + id

- The string has two parse trees:



Context-Free Grammars - Ambiguous Grammars

- A grammar is **ambiguous** if it can derive a string with two different parse trees.
 - Equivalently, there is more than one right-most or left-most derivation for some string.
- Compilers do use parse trees to derive meaning:
 - $2 * 3 + 4$



- Ambiguous grammars are problematic for compiling.
 - leave meaning of some programs ill-defined

Context-Free Grammars - Ambiguous Grammars

- We need to handle ambiguity.
- Most direct method: transform ambiguous grammars to unambiguous grammars.

1. $E \rightarrow E * E$

2. $E \rightarrow E / E$

3. $E \rightarrow E + E$

4. $E \rightarrow E - E$

5. $E \rightarrow \text{id}$

6. $E \rightarrow \text{num}$

7. $E \rightarrow (E)$

Grammar 3.5.

- We want:
 - $*$ binds tighter than $+$
 - each operator associates to the left
 - $1 - 2 - 3$ should be $(1 - 2) - 3$ instead of $1 - (2 - 3)$
- How?
 - introduce new non-terminals, making some productions used later than some other productions

Context-Free Grammars - Ambiguous Grammars

- Ambiguous grammar:

1. $E \rightarrow E * E$

2. $E \rightarrow E / E$

3. $E \rightarrow E + E$

4. $E \rightarrow E - E$

5. $E \rightarrow \text{id}$

6. $E \rightarrow \text{num}$

7. $E \rightarrow (E)$

Grammar 3.5.

- Unambiguous grammar

- E, T, F stands for **expression**, **term** and **factor**

1. $E \rightarrow E + T$

2. $E \rightarrow E - T$

3. $E \rightarrow T$

4. $T \rightarrow T * F$

5. $T \rightarrow T / F$

6. $T \rightarrow F$

7. $F \rightarrow \text{id}$

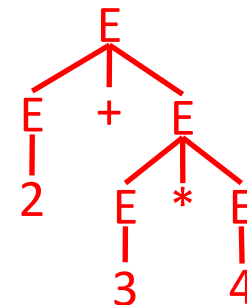
8. $F \rightarrow \text{num}$

9. $F \rightarrow (E)$

Grammar 3.8.

- Why can we enforce the **precedence**

- $2 + 3 * 4$



Context-Free Grammars - Ambiguous Grammars

- Ambiguous grammar:

1. $E \rightarrow E * E$

2. $E \rightarrow E / E$

3. $E \rightarrow E + E$

4. $E \rightarrow E - E$

5. $E \rightarrow \text{id}$

6. $E \rightarrow \text{num}$

7. $E \rightarrow (E)$

Grammar 3.5.

- Unambiguous grammar

- E, T, F stands for **expression**, **term** and **factor**

1. $E \rightarrow E + T$

2. $E \rightarrow E - T$

3. $E \rightarrow T$

4. $T \rightarrow T * F$

5. $T \rightarrow T / F$

6. $T \rightarrow F$

7. $F \rightarrow \text{id}$

8. $F \rightarrow \text{num}$

9. $F \rightarrow (E)$

Grammar 3.8.

- Why can we enforce the **left-association**?
- $X \rightarrow X \text{ op } Y$ instead of $X \rightarrow Y \text{ op } X$

Context-Free Grammars - Ambiguous Grammars

- Grammar:

1. $E \rightarrow E + T$

2. $E \rightarrow E - T$

3. $E \rightarrow T$

4. $T \rightarrow T * F$

5. $T \rightarrow T / F$

6. $T \rightarrow F$

7. $F \rightarrow \text{id}$

8. $F \rightarrow \text{num}$

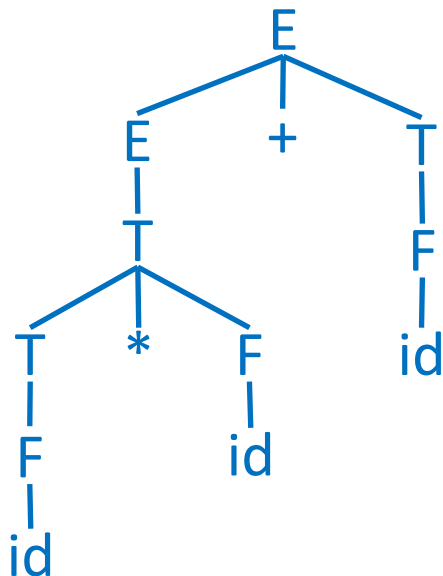
9. $F \rightarrow (E)$

Grammar 3.8.

- String:

id * id + id

- Parse Tree:



Why can we enforce the **precedence** and the **left-association**?

- Precedence**: the operators with higher precedence can only be derived later.
- Left-association**: use left recursion (the first symbol in the right-hand is the same as the symbol in the left-hand)

How about right-association?

Context-Free Grammars - Ambiguous Grammars

- There are some languages that have ambiguous grammars but no unambiguous grammar, such languages may be problematic as programming languages

Context-Free Grammars – EOF Marker

- $\$$: end of file (EOF)
- To indicate that $\$$ must come after a complete S -phrase
 - add a new start symbol S' and a new production $S' \rightarrow S\$$

$S \rightarrow E\$$

$E \rightarrow E + T$

$E \rightarrow E - T$

$E \rightarrow T$

$T \rightarrow T * F$

$T \rightarrow T / F$

$T \rightarrow F$

$F \rightarrow \text{id}$

$F \rightarrow \text{num}$

$F \rightarrow (E)$

Grammar 3.10.